

# PRANKGUARD

Sécurité adaptative par vision par ordinateur  
*Document de référence complet*

Architecture · Plan de développement · Instructions Claude Code  
Projet EPITECH — Cours de Sécurité

**Version 3.0 — Mars 2026**

Mises à jour : WinML, Licence InsightFace, Note 3DDFA\_V2

| Information            | Détail                             |
|------------------------|------------------------------------|
| Auteur                 | Alterax (EPITECH)                  |
| Repository             | github.com/alterax-code/prankguard |
| Langage                | Python 3.13 (fallback 3.12)        |
| OS cible               | Windows 10 / 11 (64 bits)          |
| Outil de développement | Claude Code (Claude Opus 4.6)      |
| Dernière mise à jour   | Mars 2026 — v3.0                   |

# 1. Contexte et objectif du projet

## 1.1 Origine du projet

PrankGuard est né d'une situation concrète à EPITECH : les référents pédagogiques s'amusent à piéger les étudiants qui laissent leur PC déverrouillé sans surveillance. L'idée est de retourner la blague en créant un programme qui détecte automatiquement quand une personne non autorisée tente de regarder ou d'utiliser le PC, et verrouille instantanément l'écran.

Le projet a évolué d'un simple script de détection faciale (v1) vers une application complète de sécurité (v18), puis vers cette architecture professionnelle multi-agent conçue pour être performante, adaptative et distribuable.

## 1.2 Ce que fait PrankGuard

PrankGuard est une application de sécurité qui tourne en arrière-plan sur n'importe quel PC Windows. Elle combine deux systèmes de protection complémentaires :

### Système 1 — Protection par vision (webcam) :

- Reconnaissance faciale : le programme apprend le visage du propriétaire et distingue les inconnus
- Détection d'intention : il ne suffit pas qu'un inconnu soit visible, il faut qu'il regarde activement l'écran pour déclencher le verrouillage
- Distinction passage vs menace : quelqu'un qui passe derrière ne déclenche rien, quelqu'un qui s'approche et fixe l'écran déclenche le lock
- Shoulder surfing : si le propriétaire est présent ET qu'un inconnu regarde par-dessus son épaule, une alerte sonore est émise sans verrouiller

### Système 2 — Protection par surveillance matérielle :

- Détection de nouveaux périphériques : toute connexion USB, réseau, Bluetooth, écran, audio ou imprimante non attendue déclenche un verrouillage immédiat
- Blocage USB : en mode verrouillé, les ports USB de stockage sont désactivés via le registre Windows pour empêcher le vol de données
- Deux modes de blocage : Desktop (bloque le stockage USB uniquement, garde clavier/souris) et Laptop (bloque tous les USB car le clavier est intégré)

## 1.3 Objectifs techniques

L'application doit répondre à ces critères simultanément :

| Objectif      | Critère de réussite                    | Pourquoi c'est important                                   |
|---------------|----------------------------------------|------------------------------------------------------------|
| Performance   | Moins de 10% CPU en moyenne            | Doit tourner en arrière-plan pendant le travail quotidien  |
| Compatibilité | Fonctionne du i3/8GB au i9/RTX         | Les étudiants EPITECH ont des machines très variées        |
| Réactivité    | Verrouillage en moins de 4 secondes    | Assez rapide pour empêcher la lecture de données sensibles |
| Fiabilité     | Zéro faux négatif prioritaire          | Mieux vaut verrouiller trop que pas assez                  |
| Vie privée    | 100% local, aucune donnée cloud        | Conforme RGPD, acceptable en entreprise                    |
| Distribuable  | Installeur .exe pour utilisateur final | N'importe qui doit pouvoir l'installer sans Python         |

## 1.4 Ce que PrankGuard ne fait PAS

Pour être clair sur le périmètre du projet :

- Ce n'est PAS un antivirus — il ne détecte pas les malwares
- Ce n'est PAS un outil de surveillance — il ne stocke aucune image ni vidéo
- Ce n'est PAS un remplacement de Windows Hello — Hello gère la présence, PrankGuard gère l'intention
- Ce n'est PAS un outil réseau — toute la détection est locale, via la webcam et les ports physiques

## 2. Architecture globale

### 2.1 Principe fondamental : l'architecture 2 phases

Le cœur de PrankGuard repose sur une idée simple : ne consommer des ressources que quand c'est nécessaire. L'application passe 90% de son temps en veille ultra-légère et ne s'active que quand elle détecte du mouvement devant la caméra.

#### Phase VEILLE — Le gardien silencieux

En phase de veille, seul un algorithme très léger tourne : MOG2 (Mixture of Gaussians v2). Cet algorithme d'OpenCV modélise chaque pixel de l'image comme un mélange de distributions statistiques. Il apprend ce qu'est le "fond" normal de la scène et détecte toute différence significative.

#### Pourquoi MOG2 et pas un simple comparateur d'images ?

- MOG2 s'adapte aux changements progressifs de lumière : le soleil qui se déplace, un nuage qui passe, un écran qui change de luminosité
- MOG2 ignore les micro-mouvements parasites : un rideau qui bouge légèrement, le reflet d'un ventilateur
- MOG2 consomme moins de 1% de CPU — c'est négligeable même sur un i3
- MOG2 est un standard documenté d'OpenCV, maintenu et optimisé depuis des années

Dès que MOG2 détecte un mouvement significatif (une personne qui entre dans le champ de vision), la phase ACTIVE se déclenche.

#### Phase ACTIVE — L'analyse complète

En phase active, tous les agents d'analyse se mettent en marche simultanément :

1. InsightFace identifie si le visage détecté est celui du propriétaire ou d'un inconnu
2. MediaPipe FaceMesh analyse la direction du regard et l'orientation de la tête
3. Le Trajectory Agent mesure si la personne s'approche ou s'éloigne
4. Le Decision Agent combine ces informations pour décider de verrouiller ou non

Cette phase consomme entre 8 et 15% de CPU selon la machine, mais elle ne dure que quelques secondes à chaque activation.

#### Phase RETOUR VEILLE — La désactivation automatique

Si aucun mouvement n'est détecté pendant 3 secondes consécutives, tous les agents lourds se désactivent et l'application retourne en phase de veille. Ce cycle est entièrement automatique et invisible pour l'utilisateur.

| Phase         | Technologie                          | CPU utilisé | Déclencheur                 | Durée typique     |
|---------------|--------------------------------------|-------------|-----------------------------|-------------------|
| VEILLE        | OpenCV MOG2                          | < 1%        | Permanent (état par défaut) | 90% du temps      |
| ACTIF         | InsightFace + MediaPipe + Trajectory | 8–15%       | Mouvement détecté par MOG2  | Quelques secondes |
| RETOUR VEILLE | Désactivation automatique            | < 1%        | 3 secondes sans mouvement   | Instantané        |

## 2.2 Architecture multi-agent

Chaque fonctionnalité de PrankGuard est encapsulée dans un fichier Python indépendant appelé "agent". Chaque agent a son propre thread (fil d'exécution), ce qui signifie qu'ils fonctionnent en parallèle sans se bloquer mutuellement.

### Pourquoi cette approche ?

- Modularité : chaque agent peut être développé, testé et débogué indépendamment
- Résilience : si un agent plante (ex: MediaPipe ne charge pas), les autres continuent de fonctionner
- Scalabilité : on peut ajouter de nouveaux agents sans toucher aux existants
- Développement parallèle : Claude Code peut travailler sur plusieurs agents en même temps via ses sous-agents

### Les 11 agents du système :

| Fichier                   | Rôle détaillé                                                                                                                                             | Thread          | Profil requis |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|---------------|
| hardware_profiler.py      | Analyse le matériel au démarrage : CPU, RAM, GPU. Exécute un benchmark ONNX pour choisir le meilleur backend. Détermine le profil de performance.         | Démarrage seul  | Tous          |
| motion_agent.py           | Capture la webcam en permanence et applique MOG2. Émet un signal quand du mouvement est détecté. C'est le déclencheur de la phase active.                 | Permanent       | Tous          |
| face_recognition_agent.py | Utilise InsightFace buffalo_sc pour identifier les visages. Compare chaque visage détecté aux encodings du propriétaire. Retourne owner/stranger/unknown. | Actif seulement | Tous          |
| gaze_estimation_agent.py  | Utilise MediaPipe FaceMesh (468 landmarks) pour calculer le vecteur de regard. Détermine si la personne regarde vers l'écran.                             | Actif seulement | BALANCED+     |

|                     |                                                                                                                                                      |                 |      |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|------|
| head_pose_agent.py  | Calcule l'orientation 3D de la tête (Yaw, Pitch, Roll) via MediaPipe + OpenCV solvePnP. Confirme si la tête est orientée vers la caméra.             | Actif seulement | Tous |
| trajectory_agent.py | Compare la taille de la bounding box entre frames successifs. Une bbox qui grandit = la personne s'approche. Ultra-léger.                            | Actif seulement | Tous |
| decision_agent.py   | Reçoit les résultats de tous les agents et prend la décision finale : rien faire, alerter, ou verrouiller. Logique détaillée en section 3.           | Actif seulement | Tous |
| device_monitor.py   | Surveille les connexions matérielles via WM_DEVICECHANGE (instantané) et WMI polling (complet). Gère la whitelist et le cooldown anti-faux positifs. | Permanent       | Tous |
| auto_throttle.py    | Monitore le CPU global toutes les 2 secondes. Si le CPU est surchargé par d'autres applications, réduit automatiquement la charge de PrankGuard.     | Permanent       | Tous |
| gui.py              | Interface graphique CustomTkinter avec thème sombre. 4 onglets : caméra live, logs, paramètres, enrollment. Gère tous les contrôles utilisateur.     | Principal       | Tous |
| prankguard.py       | Point d'entrée. Lance tous les agents, gère leur cycle de vie, et orchestre la communication entre eux.                                              | Principal       | Tous |

*Note : "Profil requis" indique le profil de performance minimum pour que l'agent soit actif. Les profils sont expliqués en section 5.*

## 3. Logique de décision

### 3.1 Principe fondamental

La logique de décision est conçue avec un principe strict : les agents ne sont JAMAIS combinés par addition de scores. Chaque agent est évalué indépendamment, et la reconnaissance faciale est prioritaire sur tout le reste.

#### Flux de décision simplifié :

5. Le face recognition reconnaît-il le propriétaire ?
  - OUI → STOP immédiat. Aucun verrouillage possible. Aucun autre agent n'est consulté. C'est la règle absolue.
  - NON (inconnu ou incertain) → passer à l'étape 2
6. Le propriétaire est-il AUSSI visible (en plus de l'inconnu) ?
  - OUI → mode SHOULDER SURFER : alerte sonore, PAS de verrouillage
  - NON → passer à l'étape 3
7. L'inconnu regarde-t-il activement l'écran ?
  - Gaze estimation : le regard est-il dirigé vers l'écran ?
  - Head pose : la tête est-elle orientée vers la caméra (Yaw < 25°, Pitch < 20°) ?
  - Trajectory : la personne s'approche-t-elle de l'écran ?
  - Si au moins 2 de ces 3 conditions sont remplies → THREAT détecté
8. Attendre le délai de confirmation (4 secondes) puis verrouiller le PC

 **IMPORTANT** : La reconnaissance du propriétaire annule TOUJOURS la menace, même si un inconnu est présent. Si le propriétaire est reconnu dans le même frame, aucun verrouillage ne se produit.

### 3.2 Tableau complet des situations

PrankGuard fonctionne dans deux modes configurables par l'utilisateur :

- Mode PÉDAGO (par défaut) : verrouille uniquement sur menace directe. Adapté pour les étudiants et le quotidien.
- Mode SECURE : verrouille aussi quand personne n'est devant le PC pendant trop longtemps. Adapté pour les environnements sensibles.

| Situation                        | Ce qui se passe concrètement                            | Mode PÉDAGO     | Mode SECURE     |
|----------------------------------|---------------------------------------------------------|-----------------|-----------------|
| Propriétaire seul devant l'écran | L'owner est reconnu, il travaille normalement           | ✅ Rien          | ✅ Rien          |
| Inconnu regarde l'écran (THREAT) | Un stranger est détecté, il fixe l'écran, il est proche | 🔒 Lock après 4s | 🔒 Lock après 4s |

|                                   |                                                                    |                  |                            |
|-----------------------------------|--------------------------------------------------------------------|------------------|----------------------------|
| Personne devant (IDLE)            | Aucun visage détecté, le PC est abandonné                          | ✓ Rien           | 🔒 Lock après 10s           |
| Quelqu'un passe (PASSING)         | Un visage apparaît brièvement au loin, sans fixer l'écran          | ✓ Rien           | 🔒 Lock après 10s cumulatif |
| Owner + inconnu (SHOULDER SURFER) | Le propriétaire est là ET un inconnu regarde par-dessus son épaule | ⚠ Alerte sonore  | ⚠ Alerte sonore            |
| Nouveau périphérique branché      | Un USB, un câble réseau, un dongle BT est connecté                 | 🔒 Lock immédiat  | 🔒 Lock immédiat            |
| THREAT mais owner visible aussi   | Un inconnu fixe l'écran MAIS le propriétaire est aussi détecté     | ✓ Priorité owner | ✓ Priorité owner           |

### 3.3 Délais et seuils

Tous les délais ont été testés et ajustés au fil des itérations du projet :

| Paramètre                             | Valeur                     | Pourquoi cette valeur                                                                                                   |
|---------------------------------------|----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Délai lock THREAT                     | 4 secondes                 | Assez long pour éviter les faux positifs si quelqu'un passe rapidement. Assez court pour empêcher la lecture prolongée. |
| Délai lock IDLE/PASSING (mode SECURE) | 10 secondes cumulatif      | Le timer ne se réinitialise PAS si la personne alterne entre IDLE et PASSING. Empêche le contournement.                 |
| Cooldown après déverrouillage         | 3 secondes                 | Évite le re-lock immédiat quand l'utilisateur revient et tape son mot de passe Windows.                                 |
| Tolérance face recognition            | 0.30 (strict)              | Distance minimale testée avec l'owner réel (0.318). 0.30 garantit la reconnaissance tout en rejetant les inconnus.      |
| Taille minimum du visage              | 20% de la hauteur du frame | Ignore les visages trop petits (personnes au fond de la pièce).                                                         |
| Seuil de centrage                     | 35% depuis le centre       | Le visage doit être relativement centré dans l'image pour être considéré comme regardant l'écran.                       |
| Cooldown device detection             | 5-8 secondes               | Empêche les faux positifs quand on modifie la configuration de monitoring ou quand Windows reconnecte un driver.        |

## 4. Stack technique détaillée

Cette section explique chaque technologie utilisée, pourquoi elle a été choisie, et comment elle s'intègre dans le système.

### 4.1 Face Recognition — InsightFace + ONNX Runtime

#### Qu'est-ce qu'InsightFace ?

InsightFace est une bibliothèque open-source de reconnaissance faciale développée par des chercheurs chinois. Elle propose plusieurs modèles pré-entraînés à la pointe de l'état de l'art, capables de détecter et reconnaître des visages en temps réel.

#### Modèle choisi : buffalo\_sc

Parmi les modèles disponibles, buffalo\_sc est celui qui correspond le mieux à PrankGuard :

- Il inclut uniquement la détection et la reconnaissance faciale, sans modules inutiles comme l'estimation d'âge ou de genre
- Il est 3 à 5 fois plus rapide que l'ancienne solution dlib utilisée dans les versions précédentes
- Sa précision sur le benchmark LFW (Labeled Faces in the Wild) dépasse les 98%
- Il fonctionne via ONNX Runtime, ce qui permet d'exploiter automatiquement le GPU si disponible

#### Qu'est-ce qu'ONNX Runtime ?

ONNX (Open Neural Network Exchange) est un format standard pour les modèles d'intelligence artificielle. ONNX Runtime est le moteur d'exécution de Microsoft qui fait tourner ces modèles de manière optimisée sur n'importe quel matériel : CPU, GPU NVIDIA (via CUDA), GPU AMD/Intel (via DirectML/WinML), ou même NPU.

 **MISE À JOUR v3.0 : WinML remplace DirectML comme provider GPU recommandé**

#### Qu'est-ce que WinML ? (Nouveau — Mars 2026)

Windows ML (WinML) est la nouvelle couche d'abstraction de Microsoft pour l'inférence IA sur Windows. Annoncé en mai 2025 et disponible depuis Windows 11 24H2, WinML remplace DirectML comme solution recommandée.

#### Ce que WinML change pour PrankGuard :

- Sélection automatique du backend : WinML détecte le matériel de l'utilisateur et choisit automatiquement le provider le plus rapide (CUDA pour NVIDIA, DirectML pour AMD/Intel, NPU pour les processeurs modernes, CPU en dernier recours)
- Aucune configuration manuelle : plus besoin de tester CPU vs DirectML au premier lancement, WinML le fait tout seul
- Support NPU : les processeurs récents (Intel Core Ultra, Qualcomm Snapdragon) intègrent des NPU (Neural Processing Units) spécialisés pour l'IA. WinML peut les exploiter, ce que DirectML ne faisait pas
- Performance améliorée : sur les GPU NVIDIA RTX, WinML avec TensorRT est environ 50% plus rapide que DirectML seul
- Compatibilité ascendante : DirectML reste fonctionnel en mode maintenance pour Windows 10 et les anciennes versions de Windows 11

Tableau des providers disponibles via WinML :

| Matériel de l'utilisateur        | Provider sélectionné par WinML | CPU estimé | Explication                                                                  |
|----------------------------------|--------------------------------|------------|------------------------------------------------------------------------------|
| GPU NVIDIA dédié (RTX/GTX)       | TensorRT for RTX EP            | 1–3%       | Le plus rapide. Utilise les Tensor Cores NVIDIA.                             |
| iGPU Intel / AMD (DX12)          | DirectML EP                    | 3–8%       | Accélération GPU générique. Peut être plus lent que CPU sur les petits iGPU. |
| NPU (Intel Core Ultra, Qualcomm) | NPU EP                         | < 1%       | Nouveau. Dédié à l'IA, très basse consommation. Windows 11 24H2+ requis.     |
| CPU seul                         | CPUExecutionProvider           | 8–15%      | Fallback universel. Fonctionne partout.                                      |

⚠ Sur les petits iGPU (Intel UHD Graphics bas de gamme), DirectML peut être plus lent que le CPU pur à cause du coût de transfert des données entre CPU et GPU. Le hardware profiler teste les deux et garde le plus rapide.

Compatibilité selon la version de Windows :

| Version Windows   | Provider recommandé           | Package Python à installer       |
|-------------------|-------------------------------|----------------------------------|
| Windows 11 24H2+  | WinML (sélection automatique) | pip install onnxruntime-winml    |
| Windows 11 < 24H2 | DirectML (legacy)             | pip install onnxruntime-directml |
| Windows 10        | DirectML (legacy)             | pip install onnxruntime-directml |

**NEW 4.1bis — Licence InsightFace (section ajoutée en v3.0)**

C'est un point juridique important qu'il faut comprendre avant toute distribution de PrankGuard.

Ce qui est libre

- Le code source de la bibliothèque Python InsightFace est sous licence MIT, sans aucune restriction d'usage (académique ou commercial)
- L'architecture ArcFace (la méthode de reconnaissance) est documentée publiquement dans des papiers de recherche
- ONNX Runtime est open-source (licence MIT)

### Ce qui est restreint

- Les modèles pré-entraînés (buffalo\_sc, buffalo\_l, antelopev2, etc.) sont réservés à la recherche non-commerciale
- Cela inclut les modèles téléchargés automatiquement par pip ET ceux téléchargés manuellement
- Toute distribution commerciale utilisant ces modèles nécessite une licence

### Conséquences concrètes pour PrankGuard

| Scénario d'utilisation                 | Licence requise ?              | Action à prendre                              |
|----------------------------------------|--------------------------------|-----------------------------------------------|
| Projet EPITECH (soutenance, cours)     | NON — usage académique         | Aucune action, utilisation libre              |
| Utilisation personnelle                | NON — usage non-commercial     | Aucune action                                 |
| Distribution gratuite sur GitHub       | NON — recherche/non-commercial | Ajouter une mention de licence dans le README |
| Vente du logiciel                      | OUI — usage commercial         | Contacteur InsightFace pour une licence       |
| Intégration dans un produit entreprise | OUI — usage commercial         | Contacteur InsightFace pour une licence       |

### Comment obtenir la licence commerciale

InsightFace ne publie pas de tarifs. Le processus est le suivant :

9. Rédiger un mail décrivant le projet, le modèle utilisé, le nombre d'utilisateurs estimé, et le modèle commercial envisagé
10. Pour les modèles de reconnaissance (buffalo) : envoyer à [recognition-oss-pack@insightface.ai](mailto:recognition-oss-pack@insightface.ai)
11. Pour le SDK InspireFace ou les modèles avancés : envoyer à [contact@insightface.ai](mailto:contact@insightface.ai)
12. InsightFace répond généralement sous quelques jours avec un devis personnalisé

### Alternative sans licence : entraîner son propre modèle

Le code d'entraînement ArcFace est sous licence MIT. Il est théoriquement possible d'entraîner son propre modèle ONNX avec ses propres données d'entraînement. Les poids résultants n'auraient aucune restriction de licence. Cependant, cela nécessite un dataset de visages conséquent et du

temps de calcul GPU significatif. C'est une option à considérer uniquement si la commercialisation devient sérieuse.

## 4.2 Gaze Estimation — MediaPipe FaceMesh

MediaPipe est un framework de Google conçu pour le traitement vidéo en temps réel. Son module FaceMesh détecte 468 points 3D (landmarks) sur le visage, en temps réel, avec une précision remarquable.

### Comment PrankGuard utilise FaceMesh :

13. FaceMesh identifie les 468 landmarks du visage, dont les landmarks spécifiques de l'iris (indices 468 à 477)
14. À partir de la position des iris par rapport aux yeux, un vecteur de regard 2D est calculé
15. Ce vecteur indique si la personne regarde vers l'écran (caméra) ou dans une autre direction
16. Le résultat est transmis au Decision Agent comme un boolean : `regarde_ecran` = oui/non

### Performances :

- 30+ FPS sur CPU seul, même sur des appareils mobiles bas de gamme
- Aucune dépendance GPU — fonctionne entièrement sur CPU
- Désactivé automatiquement en profil LITE pour économiser les ressources sur les machines faibles

## 4.3 Head Pose Estimation — MediaPipe + solvePnP

En complément du gaze estimation, le head pose mesure l'orientation physique de la tête en 3D :

- Yaw : rotation gauche/droite (la personne tourne-t-elle la tête vers l'écran ?)
- Pitch : rotation haut/bas (la personne baisse-t-elle la tête pour lire l'écran ?)
- Roll : inclinaison latérale (moins pertinent pour PrankGuard, mais calculé pour complétude)

Le calcul utilise les landmarks clés de MediaPipe (pointe du nez, coins des yeux, bords de la bouche) et l'algorithme solvePnP d'OpenCV qui résout le problème de perspective pour estimer la rotation 3D.

### Seuils :

- Si  $\text{Yaw} < 25^\circ$  ET  $\text{Pitch} < 20^\circ \rightarrow$  la tête est orientée vers l'écran  $\rightarrow$  suspect
- Si  $\text{Yaw} > 25^\circ$  OU  $\text{Pitch} > 20^\circ \rightarrow$  la personne regarde ailleurs  $\rightarrow$  passage innocent

#### Note technique : 3DDFA\_V2 comme alternative future

Des études cliniques (PMC, 2022) comparant OpenFace 2.0, 3DDFA\_V2 et MediaPipe ont démontré que 3DDFA\_V2 est plus robuste pour capturer la pose de tête sous de grandes rotations.

L'erreur moyenne de 3DDFA\_V2 reste inférieure à 5° quel que soit le plan de mouvement, tandis que MediaPipe perd en précision au-delà de 40° de rotation Yaw.

Pour PrankGuard, les rotations extrêmes ne sont pas le cas d'usage principal puisqu'on détecte surtout si quelqu'un regarde de face. MediaPipe est donc largement suffisant. Si une version future nécessite une détection sous tous les angles (ex: surveillance périmétrique 360°), 3DDFA\_V2 serait le remplacement naturel.

## 4.4 Trajectory Agent — Delta Bounding Box

Le Trajectory Agent est le plus simple mais l'un des plus efficaces des agents. Son principe :

17. À chaque frame, la bounding box (rectangle) entourant le visage est enregistrée
18. La taille de cette bounding box est comparée à celle du frame précédent
19. Si la bounding box grandit → la personne s'approche de l'écran → suspect
20. Si la bounding box reste stable ou diminue → la personne est immobile ou s'éloigne → passage

### Avantages :

- Aucune dépendance externe — quelques lignes de calcul OpenCV
- Consommation CPU négligeable
- Très efficace pour distinguer "quelqu'un qui passe derrière" de "quelqu'un qui se penche pour lire"

## 4.5 Motion Detection — OpenCV MOG2

Déjà expliqué en section 2.1, MOG2 est le gardien de la phase de veille. Pour rappel :

- Algorithme : `createBackgroundSubtractorMOG2()` d'OpenCV
- Consommation : < 1% CPU en permanence
- Robustesse : s'adapte aux changements de lumière progressifs, ignore les ombres
- Configurable : seuil de sensibilité ajustable (surface minimum de mouvement pour déclencher)

## 4.6 Hardware Profiler

Au tout premier lancement de PrankGuard, le Hardware Profiler s'exécute automatiquement. Il analyse la machine et détermine le profil de performance optimal. Ce processus prend 5 à 10 secondes et ne se répète pas aux lancements suivants (le résultat est sauvegardé).

### Ce que le profiler analyse :

21. CPU : nombre de cœurs, fréquence, modèle exact, présence d'instructions AVX/SSE (via `psutil` et `py-cpuinfo`)

22. RAM : quantité totale disponible
23. GPU : présence d'un GPU NVIDIA (via `torch.cuda.is_available()`), d'un iGPU Intel/AMD (via DirectX 12)
24. Benchmark ONNX : exécute le modèle InsightFace une fois sur CPU et une fois sur GPU (WinML/DirectML), chronomètre les deux, et garde le plus rapide
25. Résultat : attribue un profil (PERFORMANCE, BALANCED ou LITE) et sauvegarde dans un fichier config

## 4.7 Auto-throttle

L'auto-throttle surveille en permanence la charge CPU globale du système (pas seulement PrankGuard) et ajuste dynamiquement la charge de l'application pour ne pas gêner les autres programmes.

| CPU global du système | Réaction de PrankGuard                      | Explication                                                                             |
|-----------------------|---------------------------------------------|-----------------------------------------------------------------------------------------|
| Moins de 60%          | Profil normal, aucun ajustement             | La machine a de la marge, PrankGuard fonctionne à pleine capacité                       |
| 60% à 75%             | Analyse réduite à 1 frame sur 7             | La machine commence à être chargée, on réduit la fréquence d'analyse                    |
| 75% à 85%             | Bascule automatique en profil LITE          | La machine est très chargée, on désactive le gaze estimation et on réduit la résolution |
| Plus de 85%           | Mode minimal : face recog seul, 1/15 frames | La machine est saturée, on garde uniquement la reconnaissance faciale de base           |

## 4.8 Device Monitor

Le Device Monitor utilise deux mécanismes complémentaires pour détecter les connexions matérielles :

### Mécanisme 1 — WM\_DEVICECHANGE (instantané) :

Windows envoie un message système WM\_DEVICECHANGE chaque fois qu'un périphérique USB est branché ou débranché. PrankGuard écoute ces messages pour réagir instantanément. Ce mécanisme est très rapide (< 100ms) mais ne couvre que les périphériques USB classiques.

### Mécanisme 2 — WMI Polling (complet) :

Windows Management Instrumentation (WMI) permet de lister tous les périphériques connectés. PrankGuard interroge WMI toutes les 0.5 secondes pour détecter les changements sur les catégories suivantes :

- USB et HID : clés USB, souris sans fil, claviers, dongles
- Moniteurs : connexion HDMI, DisplayPort, VGA
- Réseau : connexion Ethernet, nouveau réseau WiFi

- Bluetooth : appairage d'un nouvel appareil
- Audio : casque, micro, haut-parleurs
- Imprimantes : imprimante réseau ou USB

### **Système anti-faux positifs :**

Un cooldown global de 5 à 8 secondes est appliqué après chaque modification de la configuration de monitoring. Cela empêche les faux positifs quand l'utilisateur active/désactive des options ou quand Windows reconnecte automatiquement un driver.

## 5. Profils de performance adaptatifs

PrankGuard adapte automatiquement son niveau d'analyse à la puissance de la machine. Trois profils sont définis, sélectionnés au démarrage par le Hardware Profiler, puis ajustés en temps réel par l'Auto-throttle.

| Paramètre            | PERFORMANCE                  | BALANCED       | LITE                 |
|----------------------|------------------------------|----------------|----------------------|
| Machine cible        | i7+ / 16GB+ / GPU dédié      | i5 / 8–16GB    | i3 / 8GB minimum     |
| Frames analysées     | 1 frame sur 3                | 1 frame sur 5  | 1 frame sur 10       |
| Résolution d'analyse | 480×360 pixels               | 320×240 pixels | 160×120 pixels       |
| Gaze estimation      | ✓ Actif (MediaPipe FaceMesh) | ✓ Actif        | ✗ Désactivé          |
| Head pose            | ✓ Actif                      | ✓ Actif        | ✓ Actif (très léger) |
| Trajectory           | ✓ Actif                      | ✓ Actif        | ✓ Actif (très léger) |
| CPU moyen journalier | 3–6%                         | 5–10%          | 5–8%                 |
| RAM estimée          | ~350 MB                      | ~300 MB        | ~200 MB              |

### Comment est choisi le profil ?

26. Au premier lancement, le Hardware Profiler analyse CPU + RAM + GPU et exécute un benchmark
27. Le résultat est sauvegardé dans un fichier de configuration local
28. Aux lancements suivants, le profil est lu depuis ce fichier (pas de re-benchmark)
29. En cours de fonctionnement, l'Auto-throttle peut temporairement descendre d'un profil si le CPU global dépasse les seuils
30. L'utilisateur peut aussi forcer un profil manuellement depuis les paramètres de l'interface

## 6. Conformité et respect de la vie privée

La conformité RGPD n'est pas optionnelle. C'est un prérequis pour toute distribution, même gratuite, en Europe. Cette section détaille exactement ce qui doit être implémenté.

### 6.1 Conformité RGPD

#### Premier lancement — Consentement :

Au tout premier lancement de PrankGuard, AVANT toute activation de la caméra ou de la surveillance, une fenêtre de consentement doit apparaître. Elle doit expliquer clairement :

- Ce que l'application fait : surveillance par webcam et monitoring des périphériques
- Ce qu'elle stocke : uniquement des vecteurs numériques (embeddings), JAMAIS d'images ou de vidéos
- Où sont stockées les données : uniquement sur la machine locale, jamais transmises sur Internet
- Comment supprimer les données : un bouton dédié dans les paramètres

L'utilisateur doit cliquer un bouton explicite "J'accepte" pour que l'application démarre.

#### Données stockées — Ce qui est gardé :

- Les encodings faciaux du propriétaire : ce sont des vecteurs de 512 nombres décimaux, chiffrés en AES-256. Ils ne permettent PAS de reconstituer un visage.
- Les logs d'activité : horodatage des événements ("Verrouillé à 14h32 — visage non reconnu"), sans aucune image.
- La configuration : profil de performance, préférences utilisateur, options de monitoring.

#### Données NON stockées — Jamais :

- Aucune image ou frame vidéo n'est enregistrée sur le disque
- Aucune donnée n'est envoyée sur Internet
- Aucune donnée n'est partagée avec des tiers
- Aucun identifiant de machine ou d'utilisateur n'est collecté

#### Droit à l'effacement :

Un bouton "Supprimer toutes mes données" dans les paramètres doit effacer immédiatement et irréversiblement tous les encodings, logs et configurations. L'application revient à l'état du premier lancement.

### 6.2 Transparence

- L'icône systray est TOUJOURS visible quand l'application tourne — il est impossible de la cacher

- Le tooltip sur l'icône indique l'état courant : VEILLE, ACTIF, PAUSÉ, ou DÉSACTIVÉ
- Un log d'activité est consultable à tout moment depuis l'onglet dédié de l'interface
- Un indicateur visuel montre si la caméra est active ou inactive

## 6.3 Sécurité des données

- Les encodings sont chiffrés localement en AES-256 via la bibliothèque Python cryptography
- L'application ne peut PAS tourner sans icône systray visible — pas de mode "espion"
- Un audit trail horodaté de tous les événements de sécurité est maintenu

## 7. Qualité logicielle et finitions

### 7.1 Installation pour l'utilisateur final

L'utilisateur final ne doit JAMAIS avoir besoin d'installer Python, pip, ou quoi que ce soit de technique. L'installation doit ressembler à celle de n'importe quel logiciel grand public :

31. L'utilisateur télécharge un fichier .exe depuis GitHub
32. Il double-clique et suit un assistant d'installation classique (type "Suivant > Suivant > Installer")
33. Au premier lancement, PrankGuard lui demande de s'enregistrer (enrollment facial)
34. L'application est prête à l'emploi

### 7.2 Robustesse

L'application doit gérer proprement tous les cas d'erreur sans planter :

- Webcam débranchée en cours de fonctionnement → l'app continue, log l'erreur, attend que la webcam revienne
- Agent qui plante (MediaPipe, InsightFace) → le watchdog thread le relance automatiquement
- Mode dégradé si un modèle IA ne charge pas → passage en mode face recognition seul, sans gaze/head pose
- Caméra non disponible au démarrage → message d'erreur clair avec instructions pour résoudre

### 7.3 Interface graphique

L'interface est construite avec CustomTkinter, une bibliothèque qui étend Tkinter avec des composants modernes au look sombre professionnel.

**Les 4 onglets :**

- Onglet Caméra : vue live de la webcam avec overlay coloré des états (vert = owner, rouge = threat, orange = passing, violet = shoulder surfer)
- Onglet Logs : historique horodaté des événements (verrouillages, alertes, connexions périphériques)
- Onglet Paramètres : mode PÉDAGO/SECURE, mode USB DESKTOP/LAPTOP, toggles pour chaque type de périphérique, seuils de tolérance, choix du profil
- Onglet Enrollment : interface pour re-enregistrer son visage (re-prendre des photos pour améliorer la reconnaissance)

**Raccourcis clavier :**

- L = verrouillage manuel immédiat
- P = pause / reprise de la surveillance
- U = déblocage manuel des ports USB

## 7.4 Multilingue

- Français et Anglais au minimum
- Les traductions sont stockées dans des fichiers JSON séparés (un par langue)
- Ajouter une langue = ajouter un fichier JSON, sans toucher au code

## 8. Installation de l'environnement de développement

Cette section est destinée au développeur (toi). Elle détaille comment installer tous les outils nécessaires pour travailler sur le code source de PrankGuard.

### 8.1 Prérequis système

| Élément                   | Requis                               | Commentaire                                                         |
|---------------------------|--------------------------------------|---------------------------------------------------------------------|
| Système d'exploitation    | Windows 10 ou 11 (64 bits)           | Windows 11 24H2+ recommandé pour WinML                              |
| Python                    | 3.13 (ou 3.12 en fallback)           | Vérifier avec <code>python --version</code>                         |
| Visual Studio Build Tools | Requis uniquement si on compile dlib | Pas nécessaire avec InsightFace qui utilise des wheels pré-compilés |
| Droits administrateur     | Pour le blocage USB                  | Le programme fonctionne sans admin mais ne peut pas bloquer les USB |
| Webcam                    | Intégrée ou USB                      | Indispensable pour la reconnaissance faciale                        |
| Git                       | Pour le versioning                   | Vérifier avec <code>git --version</code>                            |

### 8.2 Installation des dépendances Python

L'installation se fait en plusieurs étapes, dans cet ordre précis :

#### Étape 1 — InsightFace (nécessite un wheel spécial pour Python 3.13)

InsightFace n'a pas de wheel officiel pour Python 3.13. Il faut utiliser un wheel pré-compilé fourni par la communauté. Si cette étape échoue, vérifier que Python 3.13 est bien installé, ou passer à Python 3.12.

#### Étape 2 — Stack ONNX et IA

Installer le runtime ONNX adapté à la version de Windows :

- Sur Windows 11 24H2+ : installer `onnxruntime-winml` (sélection automatique du meilleur GPU/CPU)
- Sur Windows 10 ou Windows 11 plus ancien : installer `onnxruntime-directml` (accélération GPU via DirectX 12)
- Installer ensuite `mediapipe` (pour le gaze estimation et head pose)

#### Étape 3 — Bibliothèques système

- `opencv-python` : traitement d'images et vidéo, algorithme MOG2
- `customtkinter` : interface graphique moderne
- `keyboard` : détection des raccourcis clavier (L, P, U)
- `pywin32` : accès aux APIs Windows (`WM_DEVICECHANGE`, registre)

- wmi : interrogation WMI pour la détection de périphériques
- Pillow : manipulation d'images pour l'interface
- cryptography : chiffrement AES-256 des encodings
- psutil : monitoring CPU pour le hardware profiler et l'auto-throttle
- py-cpuinfo : détection du modèle de CPU et des instructions supportées

#### Étape 4 — Claude Code (optionnel, pour le développement assisté)

Claude Code est l'outil de développement d'Anthropic qui permet de donner des instructions en langage naturel à Claude Opus 4.6 pour qu'il écrive et modifie du code directement dans le projet. Installer via npm (Node.js requis).

### 8.3 Lancement du développement avec Claude Code

Une fois Claude Code installé, se placer dans le dossier du projet et lancer claude. Donner le prompt complet de la section 9 pour initialiser le développement multi-agent.

### 8.4 Lancement de l'application

#### Lancement normal :

Exécuter `python prankguard.py` depuis le dossier du projet. L'application démarre avec les droits utilisateur normaux. Le blocage USB ne sera pas disponible.

#### Lancement administrateur (recommandé) :

Pour bénéficier du blocage USB, lancer PowerShell en tant qu'administrateur, naviguer vers le dossier du projet, et exécuter `python prankguard.py`. Ou utiliser Start-Process avec le flag `-Verb RunAs`.

## 9. Prompt pour Claude Code

Ce prompt est conçu pour être donné tel quel à Claude Code (Claude Opus 4.6) au démarrage du projet. Il contient toutes les instructions nécessaires pour que Claude comprenne l'architecture, les contraintes, et les règles à respecter.

**⚠ Copier-coller ce prompt en entier dans Claude Code. Ne pas le modifier sauf pour mettre à jour les chemins de fichiers.**

Le prompt demande à Claude de :

35. Réécrire complètement PrankGuard avec l'architecture multi-agent décrite dans ce document
36. Créer les 11 fichiers agents en parallèle, chacun avec son propre thread
37. Utiliser InsightFace buffalo\_sc avec ONNX Runtime et WinML comme provider GPU par défaut, avec fallback DirectML pour Windows 10
38. Implémenter la logique de décision exacte décrite en section 3 (face recognition prioritaire, pas d'addition de scores)
39. Respecter la conformité RGPD dès le premier commit (consentement, pas d'images, chiffrement)
40. Conserver le code existant du device\_monitor et de la GUI s'il fonctionne, ne pas réécrire ce qui marche
41. Respecter les conventions Python : UTF-8, déclaration d'encodage en tête de fichier, docstrings
42. Ajouter la mention de licence InsightFace dans le README

*Le prompt exact est disponible dans la version précédente du document (v2.0) et reste inchangé, à l'exception des mises à jour WinML et licence mentionnées ci-dessus.*

## 10. Positionnement commercial

### 10.1 Marchés cibles

PrankGuard se positionne sur un créneau inexistant : la détection d'intention de surveillance. Aucun logiciel Windows grand public ne fait ça actuellement.

| Marché                 | Besoin spécifique                                                            | Argument PrankGuard                                                  |
|------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Entreprises (B2B)      | Protéger les données confidentielles sur les postes de travail en open space | 100% local, zéro cloud, auditable. Les DSI adorent.                  |
| Secteurs réglementés   | Banques, cabinets médicaux, cabinets juridiques, administrations             | Logs horodatés, certification ISO 27001 possible, RGPD natif         |
| Open space / coworking | Espaces partagés où n'importe qui peut passer derrière un écran              | Léger, discret, automatique. Pas besoin de verrouiller manuellement. |
| Grand public           | Télétravail dans les cafés, bibliothèques, transports                        | Simple à installer, pas d'abonnement, fonctionne offline             |
| OEM / Constructeurs    | Pré-installer sur les laptops professionnels en usine                        | API propre, faible footprint CPU/RAM, marque blanche possible        |

### 10.2 Avantage concurrentiel

- Windows Hello fait la présence/absence (l'utilisateur est-il devant son PC ?). PrankGuard fait l'intention de surveillance (quelqu'un essaie-t-il de lire l'écran ?). Personne ne fait ça nativement.
- 100% local : argument décisif en B2B. Les solutions cloud de surveillance font peur aux DSI et aux employés. PrankGuard ne transmet rien.
- Adaptatif : fonctionne du laptop étudiant i3/8GB au poste de travail i9/RTX. Aucun concurrent ne couvre cette amplitude.
- Modèle freemium défendable : version open source gratuite avec les fonctionnalités de base, version premium payante avec alertes avancées, reporting entreprise, et support.

### 10.3 Barrières à adresser

- Vie privée : la popup RGPD et le mode transparent sont non-négociables. Sans eux, l'application sera perçue comme un outil de surveillance.
- Risque Microsoft : Microsoft pourrait intégrer la détection d'intention dans Windows. Le différenciateur est la profondeur d'analyse et la personnalisation.
- Certification : pour les marchés réglementés (banques, médical), une certification ISO 27001 serait nécessaire à terme.

- Licence InsightFace : la commercialisation nécessite une licence pour les modèles pré-entraînés. Contacter [recognition-oss-pack@insightface.ai](mailto:recognition-oss-pack@insightface.ai). Alternative : entraîner son propre modèle.

## 11. Plan de développement ordonné

Les modules sont ordonnés par dépendances : chaque module ne dépend que de ceux qui le précèdent. Suivre cet ordre garantit un développement sans blocage.

| Priorité | Module                    | Difficulté | Dépendances                        | Description courte                                       |
|----------|---------------------------|------------|------------------------------------|----------------------------------------------------------|
| 1        | hardware_profiler.py      | Facile     | psutil, py-cpuinfo, onnxruntime    | Détecte le matériel et attribue un profil de performance |
| 2        | motion_agent.py           | Facile     | opencv                             | MOG2 en boucle, émet un signal quand mouvement détecté   |
| 3        | face_recognition_agent.py | Moyen      | insightface, onnxruntime, profiler | Identifie owner vs stranger via buffalo_sc               |
| 4        | head_pose_agent.py        | Moyen      | mediapipe, opencv                  | Calcule Yaw/Pitch/Roll via solvePnP                      |
| 5        | trajectory_agent.py       | Facile     | opencv                             | Compare les bounding boxes entre frames                  |
| 6        | gaze_estimation_agent.py  | Moyen      | mediapipe                          | Vecteur de regard via landmarks iris                     |
| 7        | decision_agent.py         | Moyen      | tous les agents                    | Logique de décision centrale (section 3)                 |
| 8        | auto_throttle.py          | Facile     | psutil                             | Monitore CPU global et ajuste la charge                  |
| 9        | device_monitor.py         | Difficile  | pywin32, wmi                       | WM_DEVICECHANGE + WMI polling + USB blocking             |
| 10       | gui.py                    | Moyen      | customtkinter, agents              | Interface 4 onglets avec overlays                        |
| 11       | prankguard.py             | Facile     | tous les modules                   | Launcher + orchestrateur de threads                      |
| 12       | RGPD + consentement       | Facile     | gui.py                             | Popup premier lancement + bouton suppression             |
| 13       | Chiffrement encodings     | Facile     | cryptography                       | AES-256 pour les fichiers d'encodings                    |
| 14       | Tests & benchmark         | Moyen      | tout                               | Tests unitaires + benchmark de performance               |

## 12. Distribution — Les 2 versions

PrankGuard sera distribué en deux versions téléchargeables, exactement comme VLC ou 7-Zip le font. L'utilisateur choisit selon sa situation.

### 12.1 Comparaison des deux versions

| Caractéristique            | Version LITE (Online)                      | Version FULL (Offline)                           |
|----------------------------|--------------------------------------------|--------------------------------------------------|
| Taille du téléchargement   | ~50 MB                                     | ~900 MB                                          |
| Internet requis            | Oui, au premier lancement uniquement       | Jamais après installation                        |
| Public cible               | Particuliers, étudiants, connexion normale | Entreprises, réseaux isolés, zones sans internet |
| Modèles IA                 | Téléchargés automatiquement (~300MB)       | Inclus dans l'installateur                       |
| Technologie de packaging   | PyInstaller                                | Inno Setup + Python portable                     |
| Temps du premier lancement | ~2 minutes (téléchargement)                | Immédiat                                         |
| Après le premier lancement | 100% offline, identique à la Full          | 100% offline                                     |
| Prérequis système          | Aucun (Python embarqué)                    | Aucun (Python embarqué)                          |

### 12.2 Version LITE — PyInstaller

#### Principe :

PyInstaller prend tout le code Python, toutes les dépendances, et l'interpréteur Python lui-même, et les empaquette dans un seul fichier exécutable. L'utilisateur télécharge un .exe de ~50MB, le lance, et au premier démarrage, les modèles IA (~300MB) sont téléchargés automatiquement et stockés dans le dossier AppData de Windows.

#### Étapes pour l'utilisateur :

43. Télécharger PrankGuard\_Setup\_Lite.exe depuis la page GitHub
44. Double-cliquer pour lancer
45. Au premier lancement : une barre de progression affiche "Téléchargement des modèles IA (300MB)..."
46. Les modèles sont stockés dans %APPDATA%\PrankGuard\models\
47. Dès le 2ème lancement : aucune connexion internet requise

### 12.3 Version FULL — Inno Setup

#### Principe :

Inno Setup est un outil gratuit et open-source pour créer des installateurs Windows professionnels. La Version FULL inclut tout : Python portable (pas besoin d'installer Python sur le système), toutes les bibliothèques, et les modèles IA pré-téléchargés. L'utilisateur obtient un installateur classique avec assistant "Suivant > Suivant > Installer".

### Étapes pour l'utilisateur :

48. Télécharger PrankGuard\_Setup\_Full.exe (~900MB)
49. Double-cliquer → assistant d'installation classique Windows
50. Choisir le dossier d'installation (par défaut Program Files\PrankGuard)
51. Installation terminée → raccourci sur le bureau et dans le menu démarrer
52. Aucune connexion internet requise, jamais

## 12.4 Points communs aux deux versions

- Auto-démarrage Windows optionnel : configurable dans les paramètres de l'application
- Désinstallateur propre : supprime tout y compris les modèles, encodings et logs
- Mise à jour automatique optionnelle : l'application vérifie les GitHub Releases au démarrage
- Signature de code recommandée : sans certificat (~100-300€/an), Windows SmartScreen affiche "Application inconnue"

⚠ Pour une présentation EPITECH, l'alerte SmartScreen est acceptable. Pour une distribution commerciale réelle, le certificat de signature de code est indispensable.

## 12.5 Plan de distribution

| Priorité | Tâche                             | Difficulté | Description                                                           |
|----------|-----------------------------------|------------|-----------------------------------------------------------------------|
| 15       | Script auto-download modèles      | Facile     | Progress bar au premier lancement, télécharge buffalo_sc dans AppData |
| 16       | Build PyInstaller Version Lite    | Moyen      | Configuration PyInstaller, tests sur machine vierge                   |
| 17       | Python portable + bundle modèles  | Moyen      | Préparer l'environnement Python portable avec toutes les dépendances  |
| 18       | Script Inno Setup Version Full    | Moyen      | Créer l'installateur avec wizard, raccourcis, désinstallateur         |
| 19       | Tests installation machine vierge | Facile     | Tester les deux versions sur un PC sans Python                        |
| 20       | Page de téléchargement GitHub     | Facile     | GitHub Releases avec description claire des deux versions             |

# Changelog

## Version 3.0 — Mars 2026

- WinML remplace DirectML comme provider GPU recommandé (section 4.1)
- Ajout section 4.1bis : Licence InsightFace complète — contacts, procédure, alternative
- Ajout note 3DDFA\_V2 comme alternative future à MediaPipe pour head pose (section 4.3)
- Mise à jour commandes d'installation avec onnxruntime-winml (section 8.2)
- Mise à jour prompt Claude Code avec instructions WinML et licence (section 9)
- Ajout barrière licence dans positionnement commercial (section 10.3)
- Refonte complète du document : détail ultra-professionnel de chaque section, pas de code, explications step-by-step

## Version 2.0 — Mars 2026 (première mise à jour)

- Ajout initial de WinML, licence InsightFace, et note 3DDFA\_V2

## Version 1.0 — Mars 2026 (document original)

- Création du document de référence complet à partir des conversations de développement
- Architecture 2 phases, multi-agent, 12 sections couvrant tout le projet

---

*Document généré pour transmission à Claude Opus 4.6 via Claude Code*

*PrankGuard — EPITECH Project — Alterax*